DOCUMENTACIÓN DE PRÁCTICAS · GIT

10 / 10 VERIFICADOS

Conflictos de código

10 ejercicios ejecutados, resueltos y **documentados con capturas reales**

Merge, rebase, cherry-pick, stash, binarios, inspección forense, mergetool y estrategias automáticas. Cada práctica incluye su script, su log y la evidencia de validación.

57 PASS · 0 FAIL | 23 capturas | Git 2.56.0 | GitHub

Git Bash — evidencia de ejecución

```
$ git merge feature
Auto-merging app.txt
CONFLICT (content): Merge conflict in app.txt
$ git status --short
UU app.txt
$ git add app.txt
$ git commit -m "merge: integra feature"
[PASS] el arbol de trabajo queda limpio
[PASS] se conserva el cambio de feature
RESULTADO: 5 PASS, 0 FAIL
```

Ivan · Octubre 2026 · Windows 11

github.com/75749748-blip/conflictos-git-ejercicios

10 ejercicios verificados

Este documento reúne la ejecución real de los diez ejercicios de la guía de conflictos de Git. Cada práctica se ejecutó en un repositorio temporal, se resolvió y se validó con comprobaciones automáticas. Las capturas que aparecen a continuación provienen de los logs de esa ejecución, no son recreaciones.

10/10	57	0	23
ejercicios ejecutados	comprobaciones PASS	fallos	capturas de sesión

Repositorio público:
https://github.com/75749748-blip/conflictos-git-ejercicios
Commit inicial: `df0706c` · Rama `main`

Entorno de prueba. Windows 11 · Git 2.56.0.windows.1 · Git Bash · GitHub CLI 2.102.0 · Chrome headless para generar las capturas. Cada script configura un usuario local de práctica y no modifica la configuración global del equipo.

Cómo se verificó

- Los diez scripts se ejecutan con `bash ejercicios/run-all.sh`; cada uno crea su repositorio temporal en `.work/`.
- Cada script genera el conflicto real, aplica la resolución y ejecuta comprobaciones con `[PASS]` / `[FAIL]`.
- El resultado se guarda en `ejercicios/logs/` y se resume en la última línea: `RESULTADO: N PASS, 0 FAIL`.
- Las capturas se generan desde esos logs con `build/make-captures.js`; el texto y las salidas son los de la sesión real.

Resultados

#	Ejercicio	Operación	Resultado	Evidencia
01	Conflicto simple en una línea	merge	5 PASS · 0 FAIL	script · log
02	Varios hunks en un mismo archivo	merge	6 PASS · 0 FAIL	script · log
03	Conflicto modificar/eliminar	merge	4 PASS · 0 FAIL	script · log
04	Conflicto añadir/añadir	merge	5 PASS · 0 FAIL	script · log
05	Conflicto binario	merge	4 PASS · 0 FAIL	script · log
06	Conflicto durante un rebase	rebase	5 PASS · 0 FAIL	script · log
07	Conflicto en un cherry-pick	cherry-pick	5 PASS · 0 FAIL	script · log
08	Conflicto al aplicar un stash	stash pop	4 PASS · 0 FAIL	script · log
09	Inspección forense y mergetool	merge + mergetool	4 PASS · 0 FAIL	script · log
10	Estrategias automáticas, rerere y merge drivers	estrategias	15 PASS · 0 FAIL	script · log

EJERCICIO 01 · MERGE

Conflicto simple en una línea

Básica | 5 PASS · 0 FAIL | 5 comprobaciones

> **Objetivo.** Identificar un conflicto UU, leer los marcadores y resolver conservando los dos cambios.

Marcadores, estado UU y resolución conservando ambos cambios.

Comandos clave

```
git switch main
git merge feature
git status --short
cat app.txt
git add app.txt
git commit -m "merge: integra feature conservando ambos cambios"
```

Git Bash — Ejercicio 01 · Detección del conflicto — 34 líneas de la sesión real

```
$ git merge feature
Auto-merging app.txt
CONFLICT (content): Merge conflict in app.txt
Automatic merge failed; fix conflicts and then commit the result.
[exit 1]

$ git status --short
UU app.txt

$ git show :1:app.txt
linea 1
linea 2
linea 3

$ git show :2:app.txt
linea 1
linea 2 (main)
linea 3

$ git show :3:app.txt
linea 1
linea 2 (feature)
linea 3

$ cat app.txt
linea 1
<<<<<<< HEAD
linea 2 (main)
||||||| 72164bc
linea 2
=======
linea 2 (feature)
>>>>>>> feature
linea 3
```

Log: ejercicios/logs/01-conflicto-simple.log · Conflicto simple en una línea — Git 2.56.0 · Windows · sesión verificada

Detección del conflicto — salida real de `01-conflicto-simple.log`

Git Bash — Ejercicio 01 · Resolución y validación

35 líneas de la sesión real

```
$ cat app.txt
linea 1
linea 2 (main)
linea 2 (feature)
linea 3

$ git add app.txt

$ git commit -m "merge: integra feature conservando ambos cambios"
[main b1abeb8] merge: integra feature conservando ambos cambios

$ git status --short

$ git diff --check

$ git log --oneline --graph --all --decorate
*   b1abeb8 (HEAD -> main) merge: integra feature conservando ambos cambios
|\
| * 8d54c7c (feature) feature: cambia la linea 2
* | e4214b7 main: cambia la linea 2
|/
* 72164bc base: crea app.txt

$ git show HEAD:app.txt
linea 1
linea 2 (main)
linea 2 (feature)
linea 3

  [PASS] el arbol de trabajo queda limpio
  [PASS] no quedan marcadores en el repositorio
  [PASS] se conserva el cambio de main
  [PASS] se conserva el cambio de feature
  [PASS] existe un commit de merge
RESULTADO: 5 PASS, 0 FAIL
```

Log: ejercicios/logs/01-conflicto-simple.log · Conflicto simple en una línea

Git 2.56.0 · Windows · sesión verificada

Resolución y validación — salida real de `01-conflicto-simple.log`

Validación automática

✓ 5 comprobaciones PASS · 0 FAIL

- el arbol de trabajo queda limpio
- no quedan marcadores en el repositorio
- se conserva el cambio de main
- se conserva el cambio de feature
- existe un commit de merge

> **Lección.** Git detiene la integración porque las dos ramas modificaron la misma línea. La decisión es humana: aquí se conservan ambas intenciones.

Evidencia: script · log completo · capturas

EJERCICIO 02 · MERGE

Varios hunks en un mismo archivo

Intermedia | 6 PASS · 0 FAIL | 6 comprobaciones

Objetivo. Resolver dos bloques de conflicto de forma independiente, con una decisión distinta en cada uno.

Dos regiones de conflicto y una decisión distinta en cada una.

Comandos clave

```
git merge feature
git status --short
git diff
cat config.ini
git add config.ini
git commit -m "merge: combina nombre y elige nivel debug"
```

Git Bash — Ejercicio 02 · Dos hunks en conflicto

38 líneas de la sesión real

```
$ git merge feature
Auto-merging config.ini
CONFLICT (content): Merge conflict in config.ini
Automatic merge failed; fix conflicts and then commit the result.
[exit 1]

$ git status --short
UU config.ini

$ git diff
diff --cc config.ini
index e1eabd0,5443e3b..0000000
--- a/config.ini
+++ b/config.ini
@@@ -1,5 -1,5 +1,11 @@@
  [app]
++<<<<<<< HEAD
 +nombre = demo-main
++||||||| 70ec72a
++nombre = demo
++=======
+ nombre = demo-feature
++>>>>>>> feature
  version = 1.0
  puerto = 8080

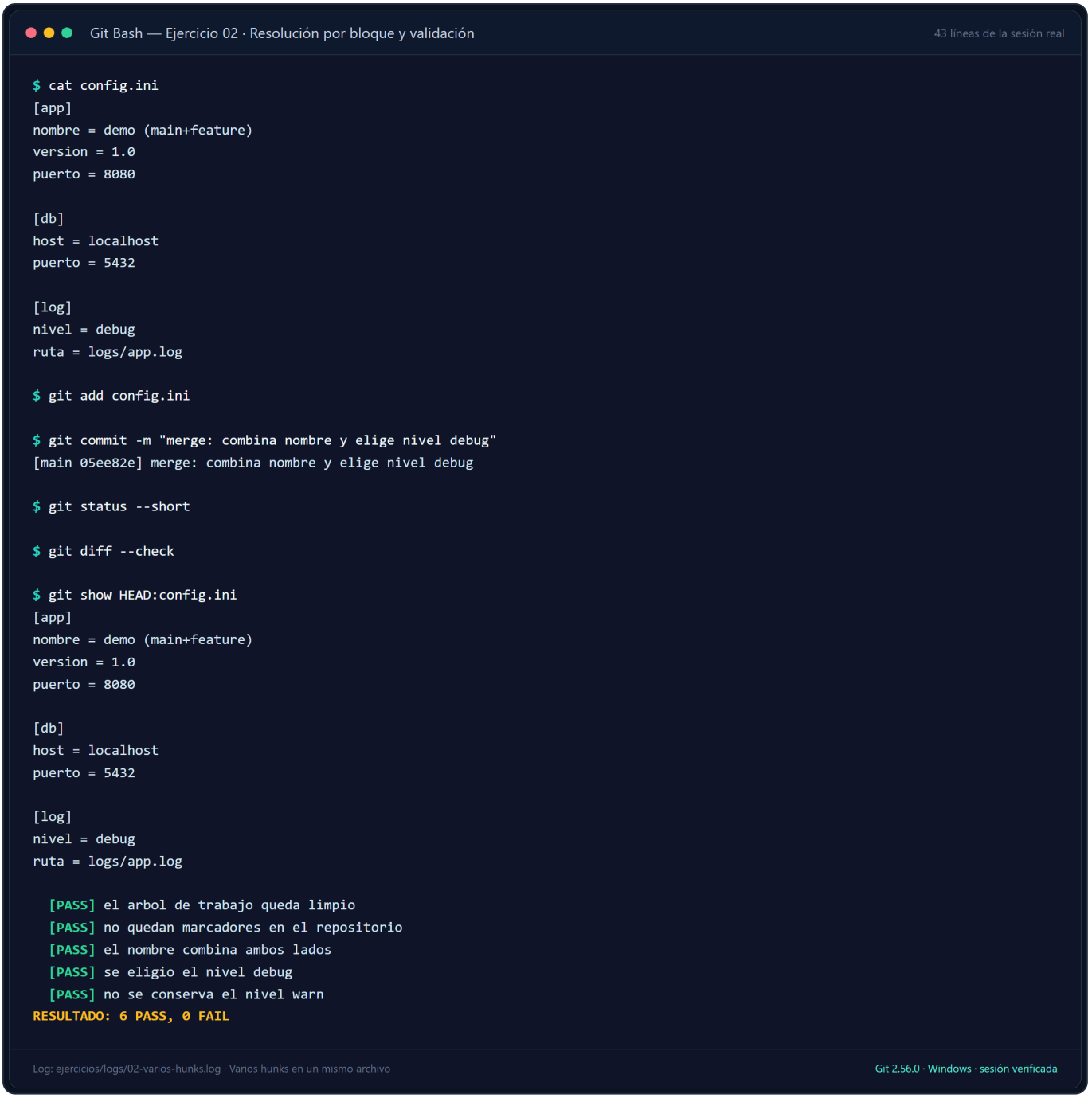
@@@ -8,5 -8,5 +14,11 @@@ host = localhos
  puerto = 5432

  [log]
++<<<<<<< HEAD
 +nivel = warn
++||||||| 70ec72a
++nivel = info
++=======
+ nivel = debug
++>>>>>>> feature
  ruta = logs/app.log
```

Log: ejercicios/logs/02-varios-hunks.log · Varios hunks en un mismo archivo

Git 2.56.0 · Windows · sesión verificada

Dos hunks en conflicto — salida real de `02-varios-hunks.log`



Resolución por bloque y validación — salida real de `02-varios-hunks.log`

Validación automática

✓ 6 comprobaciones PASS · 0 FAIL

- hay dos regiones de conflicto
- el arbol de trabajo queda limpio
- no quedan marcadores en el repositorio
- el nombre combina ambos lados
- se eligio el nivel debug
- no se conserva el nivel warn

> **Lección.** Un archivo puede contener varios conflictos. Se resuelven por separado: no conviene aplicar --ours o --theirs a todo el archivo.

Evidencia: script · log completo · capturas

EJERCICIO 03 · MERGE

Conflicto modificar/eliminar

Intermedia | 4 PASS · 0 FAIL | 4 comprobaciones

> **Objetivo.** Decidir entre conservar un archivo editado o aceptar su eliminación, probando las dos soluciones.

UD: una rama editó el archivo y la otra lo eliminó.

Comandos clave

```
git merge feature
git status --short
cat notas.txt
git add notas.txt          # Solución A: conservar
git rm notas.txt           # Solución B: eliminar
git commit -m "merge: acepta la eliminacion de notas.txt"
```

Git Bash — Ejercicio 03 · Conflicto UD y solución A: conservar — 38 líneas de la sesión real

```
$ git merge feature
CONFLICT (modify/delete): notas.txt deleted in feature and modified in HEAD.  Version HEAD of notas.txt left in tree.
Automatic merge failed; fix conflicts and then commit the result.
[exit 1]

$ git status --short
UD notas.txt

$ git status
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add/rm <file>..." as appropriate to mark resolution)
        deleted by them: notas.txt

no changes added to commit (use "git add" and/or "git commit -a")

$ cat notas.txt
uno
dos (main)
tres

$ git log --merge --oneline -- notas.txt
2e46dff main: edita notas.txt
9a4a797 feature: elimina notas.txt

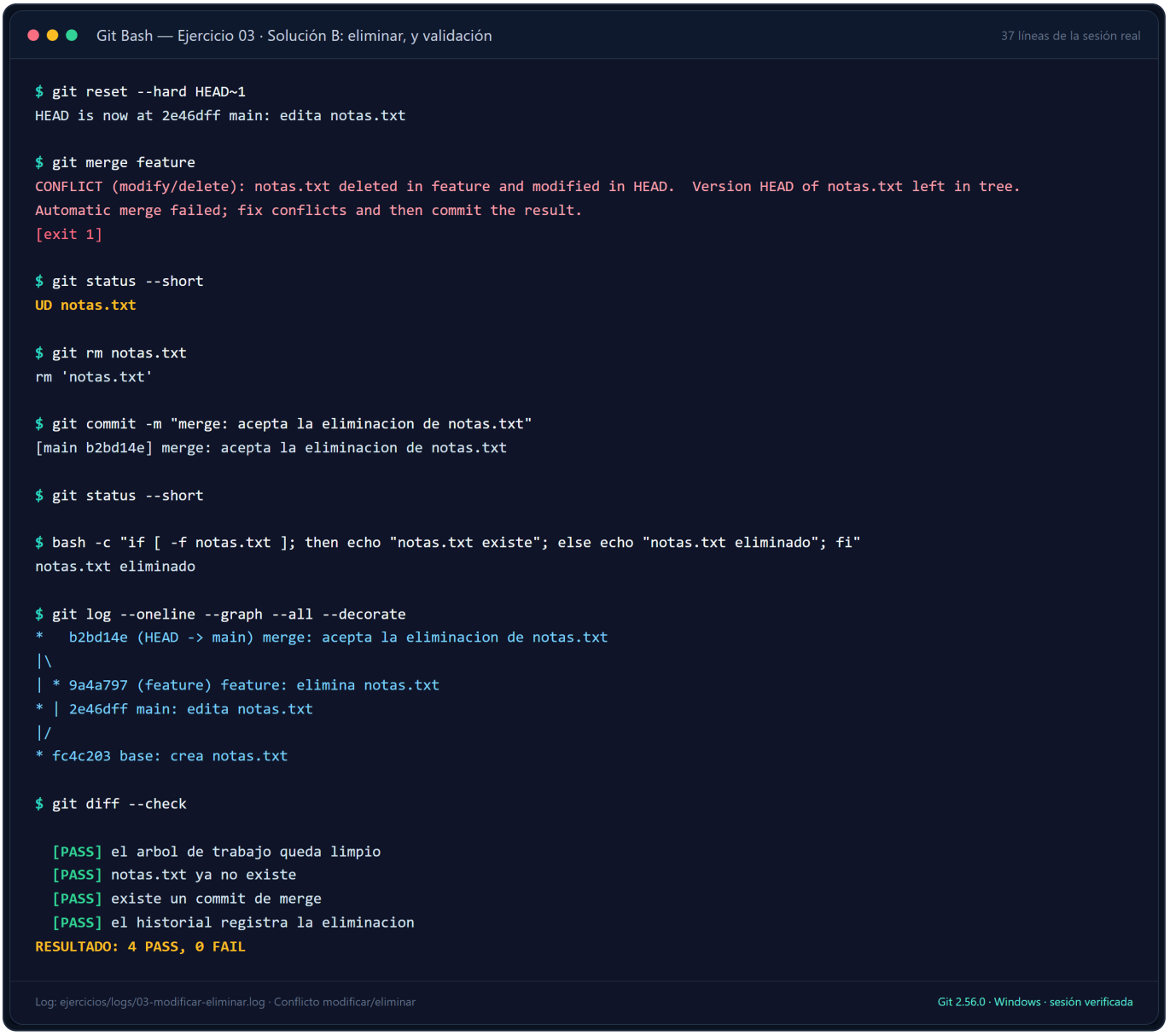
$ git add notas.txt

$ git commit -m "merge: conserva notas.txt con la edicion de main"
[main dcecb7a] merge: conserva notas.txt con la edicion de main

$ cat notas.txt
uno
dos (main)
tres
```

Log: ejercicios/logs/03-modificar-eliminar.log · Conflicto modificar/eliminar — Git 2.56.0 · Windows · sesión verificada

Conflicto UD y solución A: conservar — salida real de `03-modificar-eliminar.log`



Solución B: eliminar, y validación — salida real de `03-modificar-eliminar.log`

Validación automática

4 comprobaciones PASS · 0 FAIL

- el arbol de trabajo queda limpio
- notas.txt ya no existe
- existe un commit de merge
- el historial registra la eliminacion

> **Lección.** El estado UD obliga a una decisión de producto: ¿el archivo debe existir? Se probaron conservar y eliminar, cada una en su propio merge.

Evidencia: script · log completo · capturas

EJERCICIO 04 · MERGE

Conflicto añadir/añadir

Intermedia | 5 PASS · 0 FAIL | 5 comprobaciones

> **Objetivo.** Combinar dos archivos nuevos creados de forma independiente con el mismo nombre.

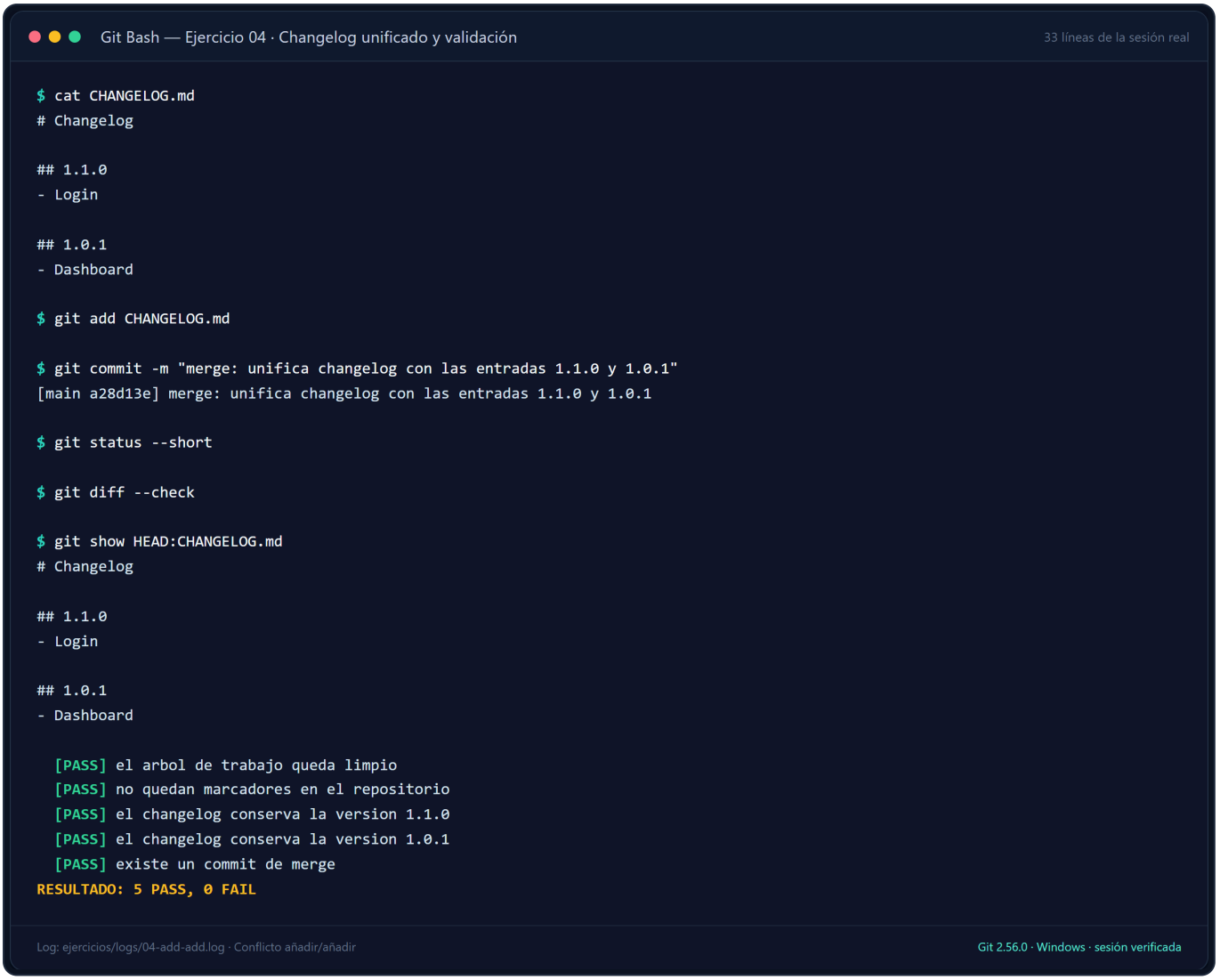
AA: dos ramas crearon el mismo archivo con contenidos distintos.

Comandos clave

```
git merge feature
git status --short          # AA CHANGELOG.md
git ls-files -u
cat CHANGELOG.md
git add CHANGELOG.md
git commit -m "merge: unifica changelog con las entradas 1.1.0 y 1.0.1"
```



Estado AA y las dos versiones — salida real de `04-add-add.log`



Changelog unificado y validación — salida real de `04-add-add.log`

Validación automática

5 comprobaciones PASS · 0 FAIL

- el arbol de trabajo queda limpio
- no quedan marcadores en el repositorio
- el changelog conserva la version 1.1.0
- el changelog conserva la version 1.0.1
- existe un commit de merge

> **Lección.** En un AA no hay una base común para el archivo. Git muestra los dos documentos completos y deja la unificación al criterio humano.

Evidencia: script · log completo · capturas

EJERCICIO 05 · MERGE

Conflicto binario

Intermedia | 4 PASS · 0 FAIL | 4 comprobaciones

> **Objetivo.** Resolver un archivo tratado como binario, donde no existen marcadores de texto.

Sin marcadores: Git no puede fusionar el contenido; se elige una versión.

Comandos clave

```
git merge feature
git status --short          # UU logo.bin
git diff                   # Cannot merge binary files
git checkout --theirs -- logo.bin
git add logo.bin
git commit -m "merge: usa la version FEATURE del logo binario"
```

Git Bash — Ejercicio 05 · Conflicto binario sin marcadores — 17 líneas de la sesión real

```
$ git merge feature
warning: Cannot merge binary files: logo.bin (HEAD vs. feature)
Auto-merging logo.bin
CONFLICT (content): Merge conflict in logo.bin
Automatic merge failed; fix conflicts and then commit the result.
[exit 1]

$ git status --short
UU logo.bin

$ git diff
diff --cc logo.bin
index 4ed3b23,2ce1430..0000000
Binary files differ

$ cat logo.bin
version MAIN
```

Log: ejercicios/logs/05-binario.log · Conflicto binario — Git 2.56.0 · Windows · sesión verificada

Conflicto binario sin marcadores — salida real de `05-binario.log`

Git Bash — Ejercicio 05 · Elección de theirs y validación

30 líneas de la sesión real

```
$ git checkout --theirs -- logo.bin

$ cat logo.bin
version FEATURE

$ git add logo.bin

$ git commit -m "merge: usa la version FEATURE del logo binario"
[main 64b0274] merge: usa la version FEATURE del logo binario

$ git status --short

$ git diff --check

$ cat logo.bin
version FEATURE

$ git log --oneline --graph --all --decorate
*   64b0274 (HEAD -> main) merge: usa la version FEATURE del logo binario
|\
| * fda3328 (feature) feature: version FEATURE del logo
* | 98194e7 main: version MAIN del logo
|/
* 505d643 base: logo v1 y atributo binario

  [PASS] el arbol de trabajo queda limpio
  [PASS] se eligio la version FEATURE
  [PASS] no se conserva la version MAIN
  [PASS] existe un commit de merge
RESULTADO: 4 PASS, 0 FAIL
```

Log: ejercicios/logs/05-binario.log · Conflicto binario

Git 2.56.0 · Windows · sesión verificada

Elección de theirs y validación — salida real de `05-binario.log`

Validación automática

4 comprobaciones PASS · 0 FAIL

- el arbol de trabajo queda limpio
- se eligio la version FEATURE
- no se conserva la version MAIN
- existe un commit de merge

> **Lección.** En binarios no se edita el conflicto: se elige una versión completa. Para conservar ambas, hay que renombrar una antes de integrar.

Evidencia: script · log completo · capturas

EJERCICIO 06 · REBASE

Conflicto durante un rebase

Avanzada | 5 PASS · 0 FAIL | 5 comprobaciones

> **Objetivo.** Resolver un conflicto durante un rebase y comprobar que la historia queda lineal, sin merge commit.

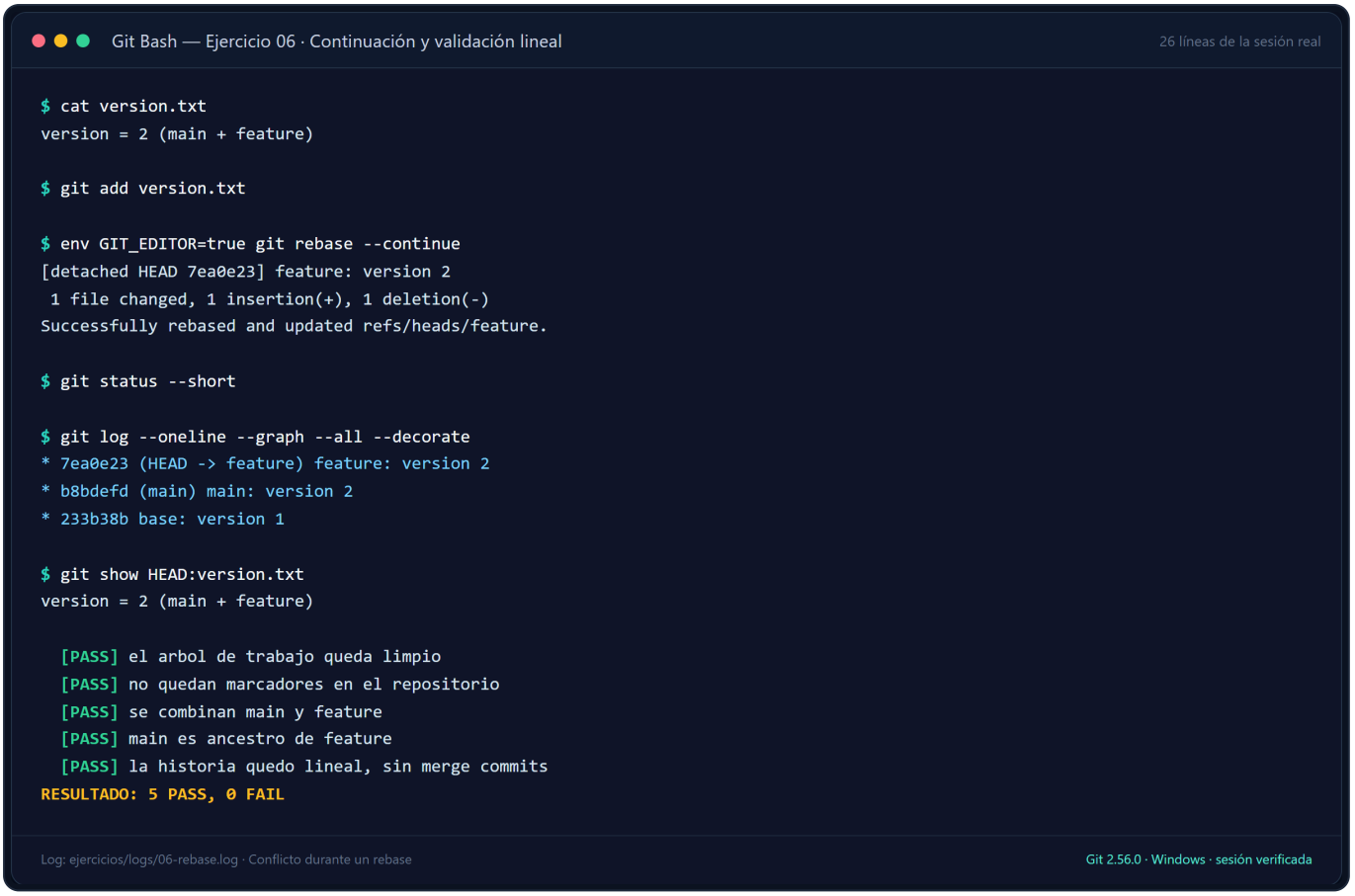
Ours y theirs se invierten: ours es la base, theirs es tu commit.

Comandos clave

```
git switch feature
git rebase main
git status
git show :2:version.txt     # main (ours en rebase)
git show :3:version.txt     # feature (theirs en rebase)
git add version.txt
GIT_EDITOR=true git rebase --continue
```



Rebase detenido y mapa invertido — salida real de `06-rebase.log`



Continuación y validación lineal — salida real de `06-rebase.log`

Validación automática

5 comprobaciones PASS · 0 FAIL

- el arbol de trabajo queda limpio
- no quedan marcadores en el repositorio
- se combinan main y feature
- main es ancestro de feature
- la historia quedo lineal, sin merge commits

> **Lección.** En rebase, ours (:2:) es la rama base y theirs (:3:) es el commit que se está reaplicando. La respuesta es la inversa de un merge.

Evidencia: script · log completo · capturas

EJERCICIO 07 · CHERRY-PICK

Conflicto en un cherry-pick

Avanzada | 5 PASS · 0 FAIL | 5 comprobaciones

> **Objetivo.** Aplicar un commit concreto de otra rama y resolver el conflicto que genera.

Aplicar un commit concreto de main sobre feature y resolver.

Comandos clave

```
HASH=$(git rev-parse main)
git switch feature
git cherry-pick $HASH
git status
git add app.txt
GIT_EDITOR=true git cherry-pick --continue
```



El commit de main no encaja en feature — salida real de `07-cherry-pick.log`

Git Bash — Ejercicio 07 · Continuación y validación

32 líneas de la sesión real

```
$ cat app.txt
linea 1
linea 2 (main + feature)
linea 3

$ git add app.txt

$ env GIT_EDITOR=true git cherry-pick --continue
[feature 1e8ea9c] main: ajusta la linea 2
 Date: Mon Oct 26 17:13:34 2026 -0500
 1 file changed, 1 insertion(+), 1 deletion(-)

$ git status --short

$ git log --oneline --graph --all --decorate
* 1e8ea9c (HEAD -> feature) main: ajusta la linea 2
* 76a2d14 feature: ajusta la linea 2
| * 75d22c0 (main) main: ajusta la linea 2
|/
* 4c0b785 base

$ git show HEAD:app.txt
linea 1
linea 2 (main + feature)
linea 3

  [PASS] el arbol de trabajo queda limpio
  [PASS] no quedan marcadores en el repositorio
  [PASS] se combinan ambos cambios
  [PASS] el commit de main quedo aplicado en feature
  [PASS] no se creo un merge commit
RESULTADO: 5 PASS, 0 FAIL
```

Log: ejercicios/logs/07-cherry-pick.log · Conflicto en un cherry-pick

Git 2.56.0 · Windows · sesión verificada

Continuación y validación — salida real de `07-cherry-pick.log`

Validación automática

✓ 5 comprobaciones PASS · 0 FAIL

- el arbol de trabajo queda limpio
- no quedan marcadores en el repositorio
- se combinan ambos cambios
- el commit de main quedo aplicado en feature
- no se creo un merge commit

> **Lección.** Cherry-pick reaplica un cambio sobre otra base. El conflicto aparece porque el parche ya no encaja; se resuelve y se continúa con --continue.

Evidencia: script · log completo · capturas

EJERCICIO 08 · STASH POP

Conflicto al aplicar un stash

Intermedia | 4 PASS · 0 FAIL | 4 comprobaciones

> **Objetivo.** Recuperar trabajo guardado temporalmente cuando el árbol ya cambió.

Updated upstream / Stashed changes: el WIP choca con un commit nuevo.

Comandos clave

```
git stash push -m "wip: linea 2"
git commit -am "main: cambia la linea 2"
git stash pop
git status --short
git add app.txt
git stash drop
git commit -m "feat: recupera el WIP combinado con el cambio de main"
```

Git Bash — Ejercicio 08 · Stash pop con conflicto

29 líneas de la sesión real

```
$ git stash pop
Auto-merging app.txt
CONFLICT (content): Merge conflict in app.txt
On branch main
Unmerged paths:
  (use "git restore --staged <file>..." to unstage)
  (use "git add <file>..." to mark resolution)
        both modified:   app.txt

no changes added to commit (use "git add" and/or "git commit -a")
The stash entry is kept in case you need it again.
[exit 1]

$ git status --short
UU app.txt

$ git stash list
stash@{0}: On main: wip: linea 2

$ cat app.txt
linea 1
<<<<<<< Updated upstream
linea 2 (commit)
||||||| Stash base
linea 2
=======
linea 2 (WIP)
>>>>>>> Stashed changes
linea 3
```

Log: ejercicios/logs/08-stash.log · Conflicto al aplicar un stash

Git 2.56.0 · Windows · sesión verificada

Stash pop con conflicto — salida real de `08-stash.log`

Git Bash — Ejercicio 08 · Resolución, drop y validación

33 líneas de la sesión real

```
$ cat app.txt
linea 1
linea 2 (commit + WIP)
linea 3

$ git add app.txt

$ git stash drop
Dropped refs/stash@{0} (9932d48a32abb505cdbe8ff47a17ddac7089bafd)

$ git commit -m "feat: recupera el WIP combinado con el cambio de main"
[main 569e045] feat: recupera el WIP combinado con el cambio de main
 1 file changed, 1 insertion(+), 1 deletion(-)

$ git status --short

$ git stash list

$ git show HEAD:app.txt
linea 1
linea 2 (commit + WIP)
linea 3

$ git log --oneline --graph --all --decorate
* 569e045 (HEAD -> main) feat: recupera el WIP combinado con el cambio de main
* d4f42c1 main: cambia la linea 2
* d0d3c21 base

  [PASS] el arbol de trabajo queda limpio
  [PASS] no quedan marcadores en el repositorio
  [PASS] se combinan commit y WIP
  [PASS] el stash se elimino despues de resolver
RESULTADO: 4 PASS, 0 FAIL
```

Log: ejercicios/logs/08-stash.log · Conflicto al aplicar un stash

Git 2.56.0 · Windows · sesión verificada

Resolución, drop y validación — salida real de `08-stash.log`

Validación automática

✓ 4 comprobaciones PASS · 0 FAIL

- el arbol de trabajo queda limpio
- no quedan marcadores en el repositorio
- se combinan commit y WIP
- el stash se elimino despues de resolver

> **Lección.** Si stash pop conflictúa, el stash no se elimina. Se resuelve, se marca el archivo, se hace drop y se confirma el resultado.

Evidencia: script · log completo · capturas

EJERCICIO 09 · MERGE + MERGETOOL

Inspección forense y mergetool

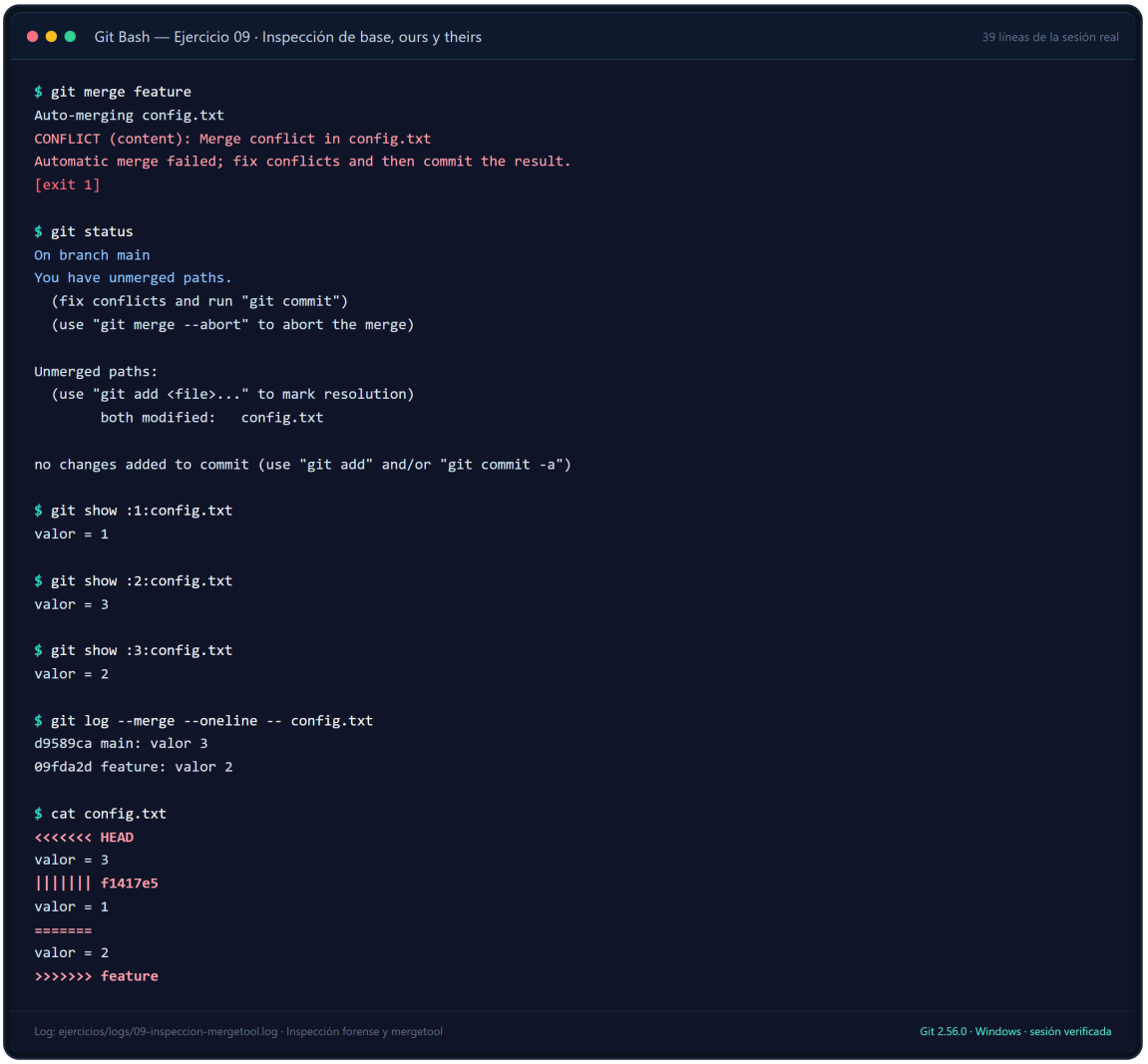
Intermedia | 4 PASS · 0 FAIL | 4 comprobaciones

> **Objetivo.** Usar las etapas del índice, el estilo diff3, git log --merge y una herramienta de merge para resolver con información completa.

Etapas :1: :2: :3:, diff3, git log --merge y una herramienta de merge.

Comandos clave

```
git merge feature
git show :1:config.txt    # base
git show :2:config.txt    # ours
git show :3:config.txt    # theirs
git log --merge --oneline -- config.txt
git mergetool --tool=resolver
git commit -m "merge: resuelve config.txt con mergetool"
```



Inspección de base, ours y theirs — salida real de `09-inspeccion-mergetool.log`

Git Bash — Ejercicio 09 · Resolución con una herramienta de merge

31 líneas de la sesión real

```
$ cat resolver.sh
#!/usr/bin/env bash
# Herramienta de merge no interactiva para la demostracion.
# Argumentos de git mergetool: BASE LOCAL REMOTE MERGED
printf 'valor = 5\n' > "$4"
exit 0

$ git config merge.tool resolver

$ git config mergetool.prompt false

$ git config mergetool.resolver.cmd "/c/Users/gambo/Documents/Git-Conflictos/.work/ejercicio-09-inspeccion/resolver.sh "$BASE"
"$LOCAL" "$REMOTE" "$MERGED""

$ git config --get mergetool.resolver.cmd
C:/Users/gambo/Documents/Git-Conflictos/.work/ejercicio-09-inspeccion/resolver.sh "$BASE" "$LOCAL" "$REMOTE" "$MERGED"

$ printf 'y\n' | git mergetool --tool=resolver
Merging:
config.txt

Normal merge conflict for 'config.txt':
  {local}: modified file
  {remote}: modified file

$ git status --short
M  config.txt
?? config.txt.orig
?? resolver.sh

$ cat config.txt
valor = 5
```

Log: ejercicios/logs/09-inspeccion-mergetool.log · Inspección forense y mergetool

Git 2.56.0 · Windows · sesión verificada

Resolución con una herramienta de merge — salida real de `09-inspeccion-mergetool.log`

Git Bash — Ejercicio 09 · Cierre y validación

23 líneas de la sesión real

```
$ git commit -m "merge: resuelve config.txt con mergetool"
[main 4cea9cb] merge: resuelve config.txt con mergetool

$ git status --short

$ git diff --check

$ git show HEAD:config.txt
valor = 5

$ git log --oneline --graph --all --decorate
*   4cea9cb (HEAD -> main) merge: resuelve config.txt con mergetool
|\
| * 09fda2d (feature) feature: valor 2
* | d9589ca main: valor 3
|/
* f1417e5 base: valor 1

  [PASS] el arbol de trabajo queda limpio
  [PASS] no quedan marcadores en el repositorio
  [PASS] el valor final es 5
  [PASS] existe un commit de merge
RESULTADO: 4 PASS, 0 FAIL
```

Log: ejercicios/logs/09-inspeccion-mergetool.log · Inspección forense y mergetool

Git 2.56.0 · Windows · sesión verificada

Cierre y validación — salida real de `09-inspeccion-mergetool.log`

Validación automática

✓ **4 comprobaciones PASS · 0 FAIL**

- el arbol de trabajo queda limpio
- no quedan marcadores en el repositorio
- el valor final es 5
- existe un commit de merge

> **Lección.** Antes de editar, reconstruye las tres versiones. La base (|||||||) explica qué cambió cada lado y evita elegir a ciegas.

Evidencia: script · log completo · capturas

EJERCICIO 10 · ESTRATEGIAS

Estrategias automáticas, rerere y merge drivers

Avanzada | 15 PASS · 0 FAIL | 15 comprobaciones

> **Objetivo.** Comparar las herramientas que reducen el trabajo manual y entender qué se conserva y qué se descarta en cada una.

-X ours, -s ours, rerere, merge=union y merge=ours, con resultados verificados.

Comandos clave

```
git merge -X ours feature       # conserva hunks no conflictivos
git merge -s ours feature       # descarta todo el árbol de feature
git config rerere.enabled true
git config merge.ours.driver true
git merge feature-union         # .gitattributes: merge=union
```

Git Bash — Ejercicio 10 · Parte A: -X ours frente a -s ours — 39 líneas de la sesión real

```
$ git merge --no-edit -X ours feature
Auto-merging app.txt
Merge made by the 'ort' strategy.
 nueva.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 nueva.txt

$ cat app.txt
linea 1
linea 2 (main)
linea 3

$ ls nueva.txt
nueva.txt

$ git show --stat --oneline HEAD
be6bbd4 Merge branch 'feature'

 nueva.txt | 1 +
 1 file changed, 1 insertion(+)

$ git reset --hard HEAD~1
HEAD is now at 619cb5d main: linea 2

$ git merge --no-edit -s ours feature
Merge made by the 'ours' strategy.

$ cat app.txt
linea 1
linea 2 (main)
linea 3

$ if [ -e nueva.txt ]; then echo 'nueva.txt existe'; else echo 'nueva.txt NO existe'; fi
nueva.txt NO existe

$ git show --stat --oneline HEAD
de56efe Merge branch 'feature'

$ git diff HEAD~1 HEAD --stat
```

Log: ejercicios/logs/10-estrategias.log · Estrategias automáticas, rerere y merge drivers — Git 2.56.0 · Windows · sesión verificada

Parte A: -X ours frente a -s ours — salida real de `10-estrategias.log`

Git Bash — Ejercicio 10 · Parte B: rerere reutiliza la resolución

57 líneas de la sesión real

```
$ git add app.txt

$ git merge feature
Auto-merging app.txt
CONFLICT (content): Merge conflict in app.txt
Recorded preimage for 'app.txt'
Automatic merge failed; fix conflicts and then commit the result.
[exit 1]

$ cat app.txt
linea 1
<<<<<<< HEAD
linea 2 (main)
=======
linea 2 (feature)
>>>>>>> feature
linea 3

$ git add app.txt

$ git rerere status
app.txt

$ git commit -m "merge: resolucion registrada por rerere"
Recorded resolution for 'app.txt'.
[main 4895465] merge: resolucion registrada por rerere

$ git reset --hard HEAD~1
HEAD is now at 3c01bdc main: linea 2

$ git merge feature
Auto-merging app.txt
CONFLICT (content): Merge conflict in app.txt
Staged 'app.txt' using previous resolution.
Automatic merge failed; fix conflicts and then commit the result.
[exit 1]

$ git status --short
M  app.txt

$ cat app.txt
linea 1
linea 2 (main + feature)
linea 3

$ git add app.txt

$ git commit -m "merge: rerere reutiliza la resolucion"
[main d1452b5] merge: rerere reutiliza la resolucion

$ git log --oneline --graph --all --decorate
*   d1452b5 (HEAD -> main) merge: rerere reutiliza la resolucion
|\
| * 99a09c0 (feature) feature: linea 2
* | 3c01bdc main: linea 2
|/
* 447b787 base
```

Log: ejercicios/logs/10-estrategias.log · Estrategias automáticas, rerere y merge drivers

Git 2.56.0 · Windows · sesión verificada

Parte B: rerere reutiliza la resolución — salida real de `10-estrategias.log`

Git Bash — Ejercicio 10 · Parte C: merge=union y merge=ours

22 líneas de la sesión real

```
$ git merge --no-edit feature-union
Auto-merging deps.lock
Merge made by the 'ort' strategy.
 deps.lock | 1 +
 1 file changed, 1 insertion(+)

$ cat deps.lock
lock-main
lock-b

$ git check-attr merge deps.lock
deps.lock: merge: union

$ git merge --no-edit feature-ours
Auto-merging config.ours
Merge made by the 'ort' strategy.

$ cat config.ours
main

$ git check-attr merge config.ours
config.ours: merge: ours
```

Log: ejercicios/logs/10-estrategias.log · Estrategias automáticas, rerere y merge drivers

Git 2.56.0 · Windows · sesión verificada

Parte C: merge=union y merge=ours — salida real de `10-estrategias.log`

Git Bash — Ejercicio 10 · Validación final de las tres partes

21 líneas de la sesión real

```
$ git status --short

$ git diff --check

$ git log --oneline --graph --all --decorate
*   afe2a9d (HEAD -> main) Merge branch 'feature-ours'
|\
| * 17ac589 (feature-ours) feature: config.ours
* | 108525f main: config.ours
|/
* 221a983 main: driver ours
*   0c44bfd Merge branch 'feature-union'
|\
| * ce9b35b (feature-union) feature: lock-b
* | c1ab437 main: lock-main
|/
* 3d2b0b8 main: lockfile con merge=union

  [PASS] el arbol de trabajo queda limpio (drivers)
  [PASS] no quedan marcadores (drivers)
RESULTADO: 15 PASS, 0 FAIL
```

Log: ejercicios/logs/10-estrategias.log · Estrategias automáticas, rerere y merge drivers

Git 2.56.0 · Windows · sesión verificada

Validación final de las tres partes — salida real de `10-estrategias.log`

Validación automática

✓ **15 comprobaciones PASS · 0 FAIL**

- -X ours conserva la linea de main
- -X ours conserva nueva.txt de feature
- -s ours conserva la linea de main
- -s ours descarta nueva.txt
- el arbol de trabajo queda limpio
- no quedan marcadores en el repositorio
- rerere aplico la resolucion combinada
- existe cache de rerere
- union conserva lock-main
- union conserva lock-b
- union no deja marcadores
- merge=ours conserva main
- merge=ours no aplica feature
- el arbol de trabajo queda limpio (drivers)
- no quedan marcadores (drivers)

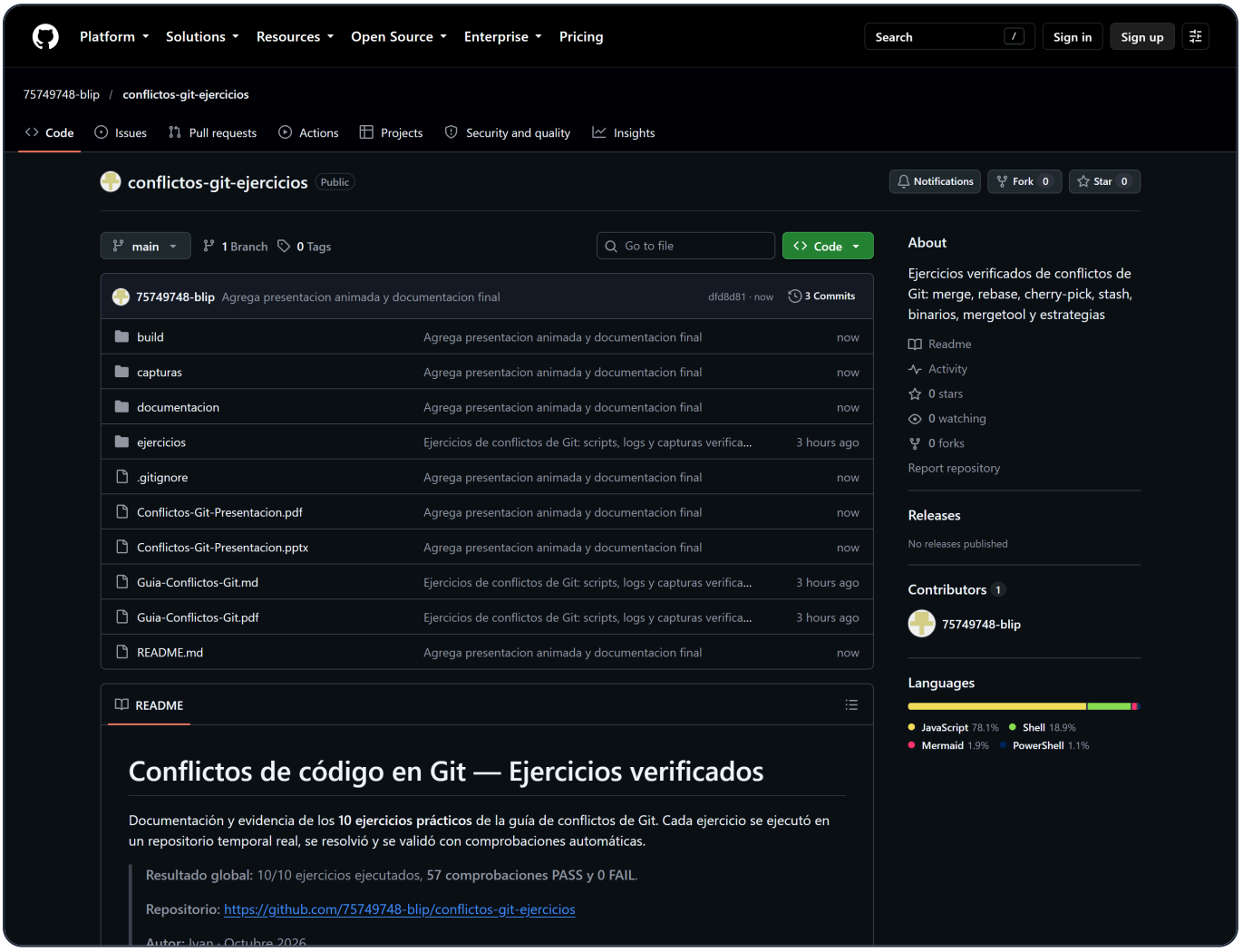
> **Lección.** Ninguna estrategia automática avisa de lo que descartó. -X ours fusiona; -s ours ignora el árbol ajeno; union concatena; ours oculta cambios.

Evidencia: script · log completo · capturas

CIERRE · EVIDENCIA DE PUBLICACIÓN

Repositorio en GitHub

Los scripts, logs, capturas, la guía y la presentación están publicados en el repositorio. La siguiente captura corresponde a la página pública después de publicar la guía, la presentación y la documentación.



URL: https://github.com/75749748-blip/conflictos-git-ejercicios
Commit inicial: df0706c · **Rama:** main · **Visibilidad:** pública

Declaración de verificación. Los diez ejercicios fueron ejecutados en este equipo con Git 2.56.0.windows.1. Los logs muestran 57 comprobaciones PASS y 0 FAIL. Las capturas se obtuvieron de esos logs. El repositorio de GitHub contiene el código y la evidencia para que cualquier persona pueda repetir el proceso.

Documento generado el 26 de octubre de 2026. Autor: Ivan. Guía base: *Detección y resolución de conflictos de código*.